ECOWASTE

Projeto DevOps — Fase: Navegando pelo Mundo DevOps
Integrante: Joice Fernanda de Souza

1. Descrição do Pipeline

Ferramenta utilizada:

- GitHub Actions (workflow `.github/workflows/ci-cd.yml`)

Objetivo do pipeline:
Automatizar o processo de build, teste e deploy da aplicação **EcoWaste**, garantindo integração contínua e entrega contínua (CI/CD).

Etapas principais do pipeline:

1. **Checkout do código:** Clona o repositório.
2. **Configuração do ambiente .NET:** Usa a ação `actions/setup-dotnet@v3` para preparar o .NET 8.0.
3. **Build:** Compila o projeto EcoWaste.Api e suas dependências.
4. **Testes automatizados:** Executa os testes do projeto `EcoWaste.Tests` usando `dotnet test`.
5. **Build da imagem Docker:** Cria a imagem da API (`ecowaste-api`) a partir do Dockerfile.
6. **Deploy:** Envia a imagem para o ambiente de produção (local com Docker Compose).

Lógica do fluxo:
O pipeline garante que **nenhuma alteração chegue ao deploy sem passar pelos testes automatizados**.
Erros de build ou teste interrompem o fluxo, assegurando estabilidade e qualidade do código.

2. Docker — Arquitetura e Comandos

Arquitetura Docker:

- O projeto utiliza **dois containers**:
  - `sqlserver`: Banco de dados SQL Server 2022
  - `ecowaste-api`: Aplicação ASP.NET Core 8.0

Rede:
Ambos estão conectados à mesma rede `ecowaste-network`, permitindo comunicação interna.

Volumes:
Um volume `sqlvolume` persiste os dados do banco.

Principais comandos utilizados:

Construir a imagem da API: docker build -t ecowaste-api .

Subir os containers em modo detached: docker-compose up -d

Verificar containers ativos: docker ps

Parar e remover os containers: docker-compose down

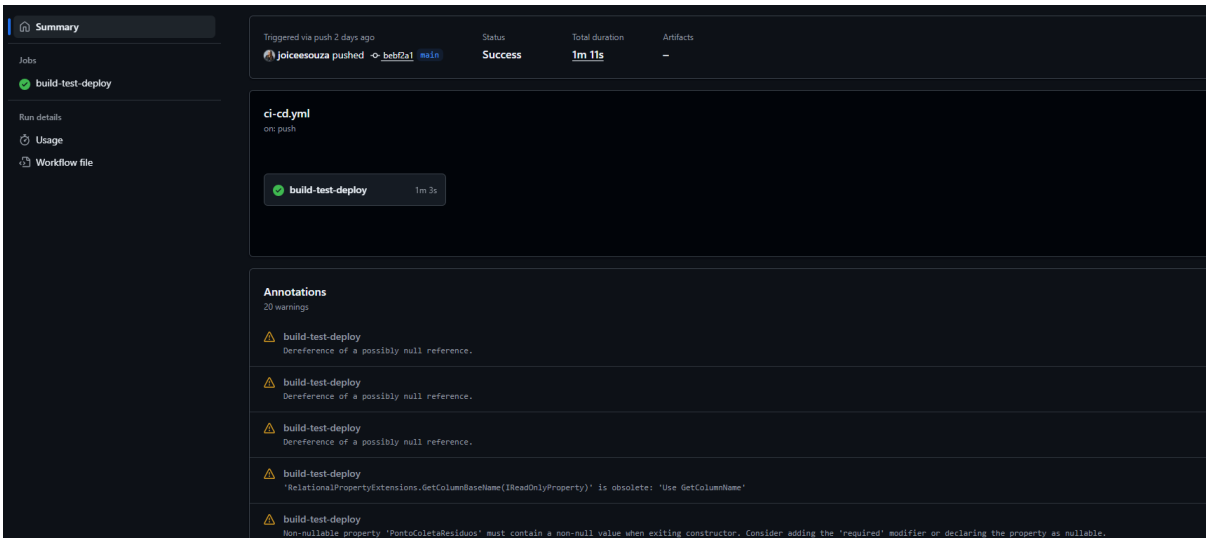
Imagem criada:

- Nome: `ecowaste-api`
- Base: `mcr.microsoft.com/dotnet/aspnet:8.0`
- Porta exposta: `8080:80`

⚙️ 3. Prints do Pipeline em Execução

📸 *Captura de tela do GitHub Actions:*

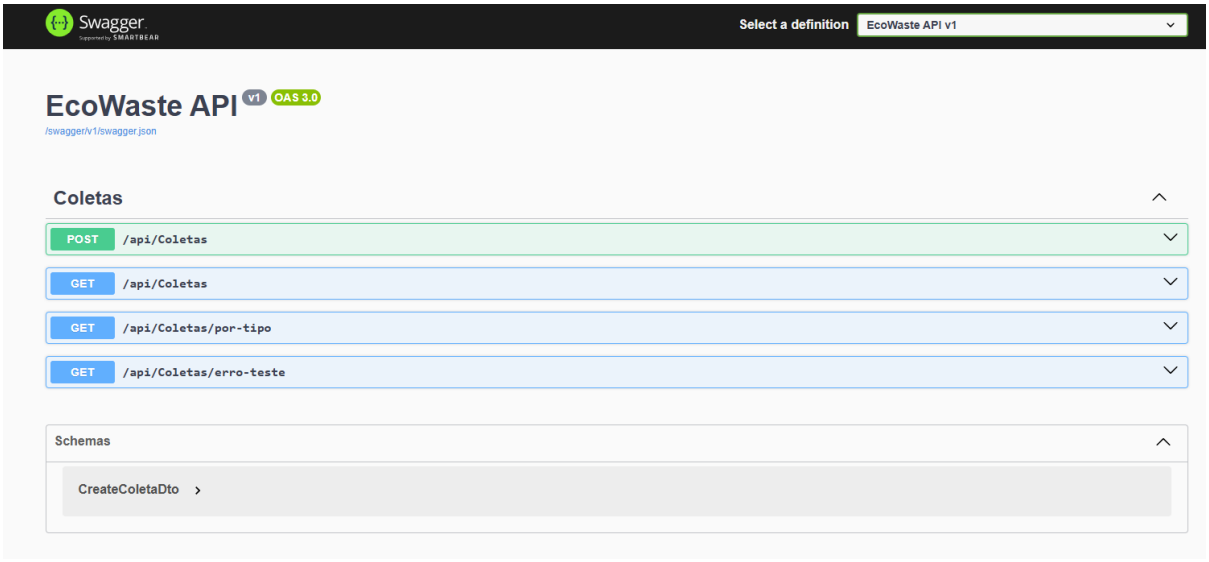
- Testes executados com sucesso
- Deploy concluído



🌍 4. Prints dos Ambientes

Ambiente Staging (local Docker Compose):

- Print do Swagger em http://localhost:8080/swagger



Ambiente Produção:

- Print do sistema rodando via Docker



- Print do log do container confirmando inicialização da API



🧩 5. Desafios e Soluções

Desafio	Solução Aplicada
Erro de build no Docker (TLS handshake timeout)	Ajuste de DNS do Docker para 8.8.8.8 e 1.1.1.1
Falha ao iniciar containers por WSL desatualizado	Atualização via comando wsl --update
Virtualização desativada na BIOS	Acesso à BIOS e ativação de “Intel Virtualization Technology”
Estrutura incorreta do workflow	Criação da pasta .github/workflows e arquivo ci-cd.yml válido
Dockerfile incompatível com .NET 9	Atualização das imagens base sdk:8.0 e aspnet:8.0

⌨️ 6. Conclusão

O pipeline do **EcoWaste** garante integração contínua e entrega automatizada da aplicação de forma confiável.

A configuração com **Docker Compose** possibilita simular ambientes reais (staging e produção),
facilitando testes, implantação e manutenção da solução.